
NAME: S.DHARSHINI

REG NO: 192424258

SUB CODE: DSA0502

SUB NAME: QUERY PROCESSING FOR DATA SCIENCE

CO2 ASSESSMENT TOOL 2

04.08.2026

---

3. Online Shopping System

Database tables include:

Customer(CustomerID, Name)

Product(ProductID, ProductName, Price)

Order(OrderID, CustomerID, OrderDate)

OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

Q2. What would happen if ProductID were stored directly in the Order table?

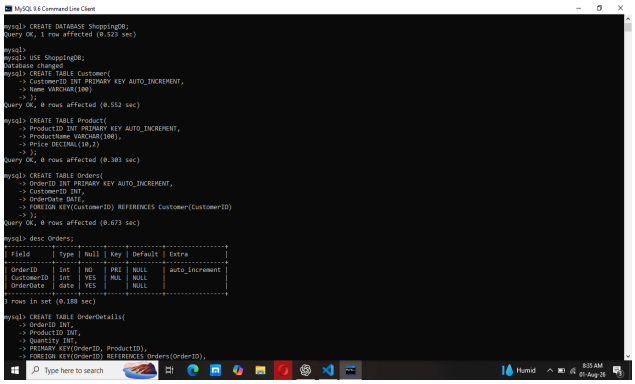
AIM: To develop an Online Shopping System using Python and MySQL to manage customers, products, orders, and order details efficiently.

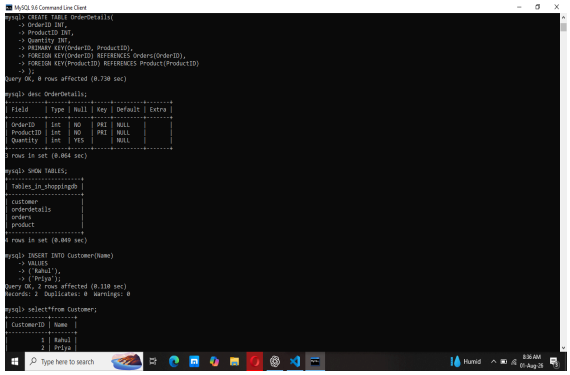
PROCEDURE:

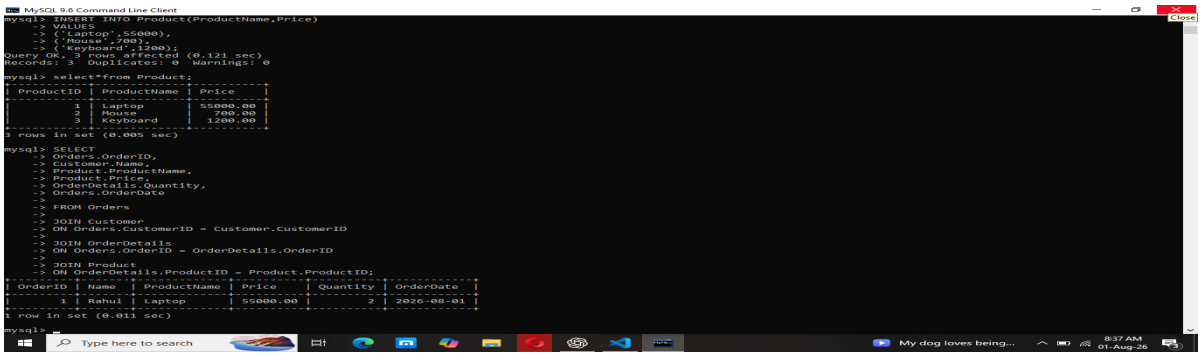
- Create the ShoppingDB database.
- Create Customer, Product, Orders, and OrderDetails tables.
- Insert sample customer and product records.
- Connect Python with MySQL.

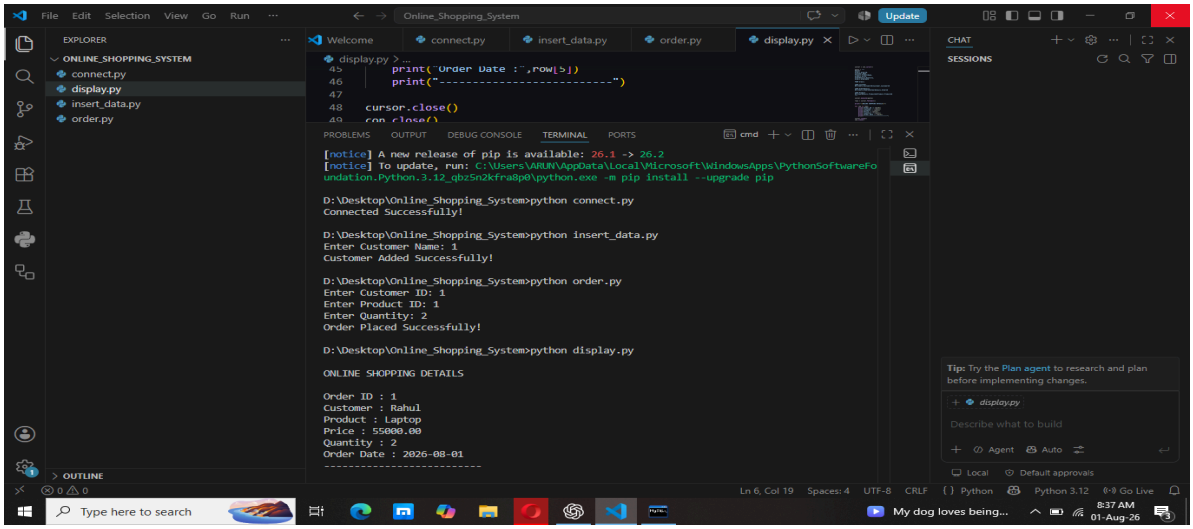
- Register a customer.
- Place an order.
- Store ordered products in the OrderDetails table.
- Display order details using SQL JOIN.
- Verify the output.

OUTPUT:









Viva Questions

Q1. Why is the OrderDetails table required in this schema?

The **OrderDetails** table is required because one order can contain multiple products, and one product can appear in many different orders. It stores the **ProductID** and **Quantity** for each product in an order, avoiding data duplication.

Q2. What would happen if ProductID were stored directly in the Order table?

If **ProductID** were stored directly in the **Orders** table, each order could contain only **one product**. To store multiple products, duplicate order records would be required, causing redundancy and making the database inefficient.

CONCLUSION: The Online Shopping System efficiently manages customers, products, and orders using a relational database. The use of the OrderDetails table allows multiple products to be stored for a single order while reducing redundancy.

8. Food Delivery Application

Tables:

Customer(CustomerID, Name)

Restaurant(RestaurantID, Name)

Order(OrderID, CustomerID, RestaurantID)

DeliveryAgent(AgentID, Name)

Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

Viva Questions

Q1. Why is Delivery maintained as a separate table?

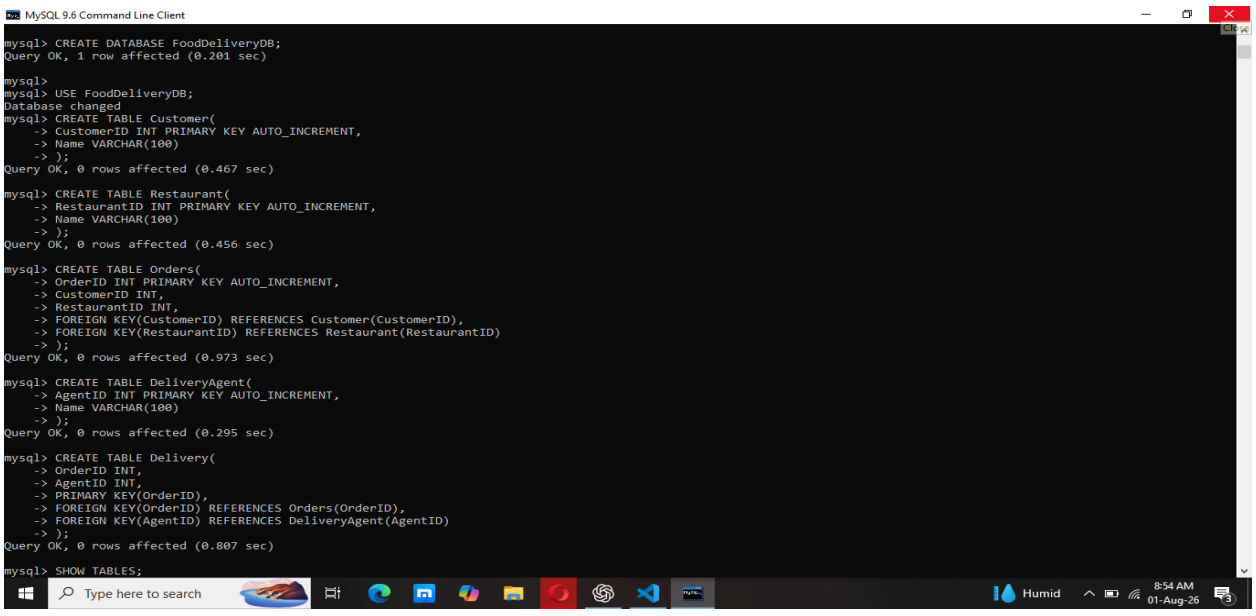
Q2. How would the schema change if multiple delivery agents could handle a single order?

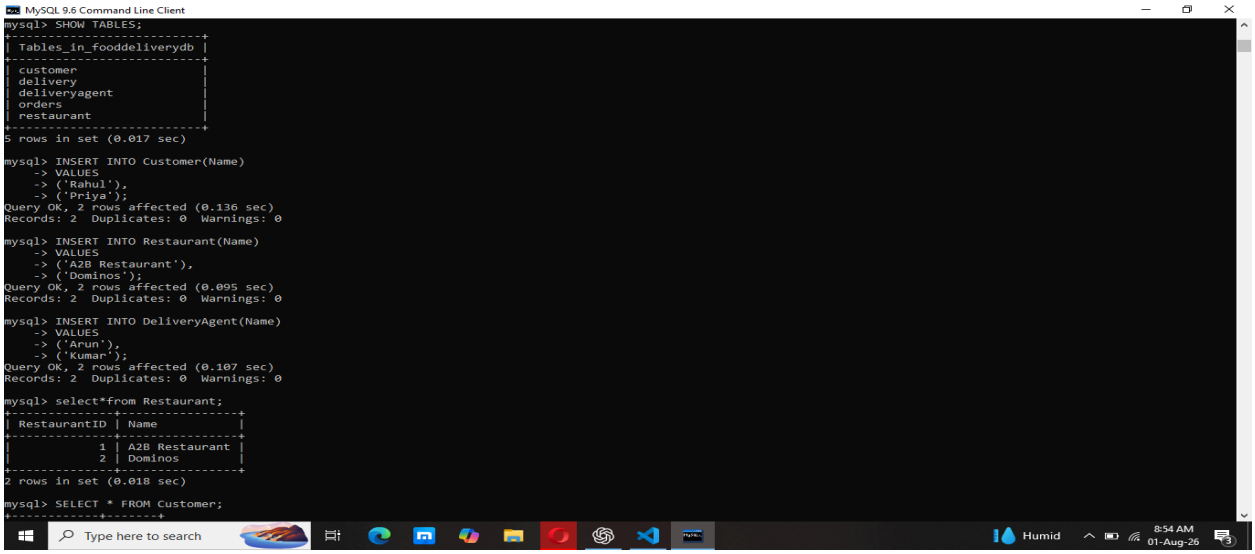
AIM: To develop a Food Delivery Application using Python and MySQL for managing customers, restaurants, food orders, and delivery agents efficiently.

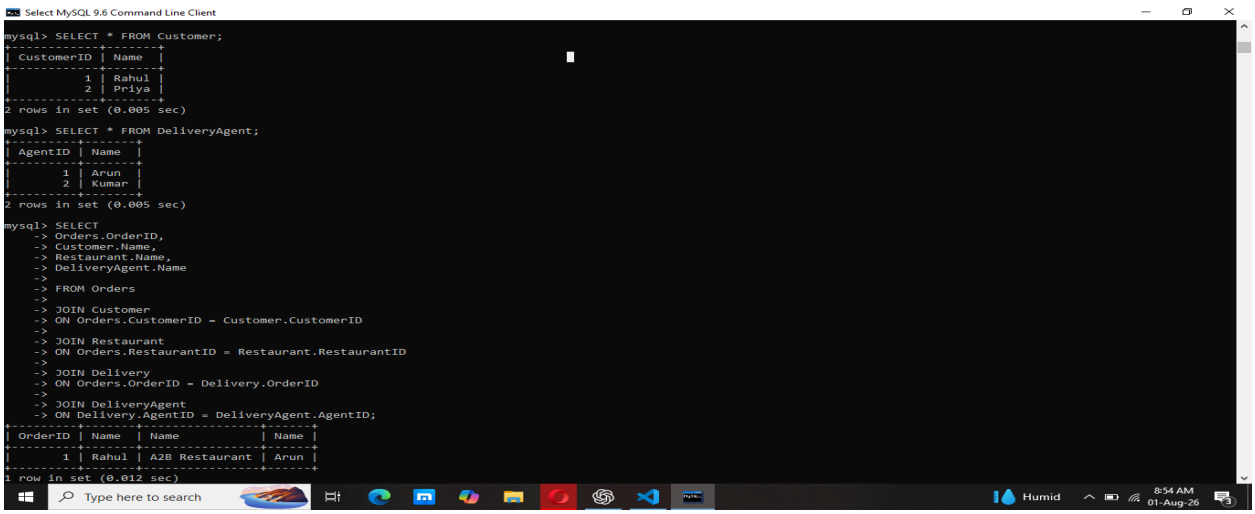
PROCEDURE:

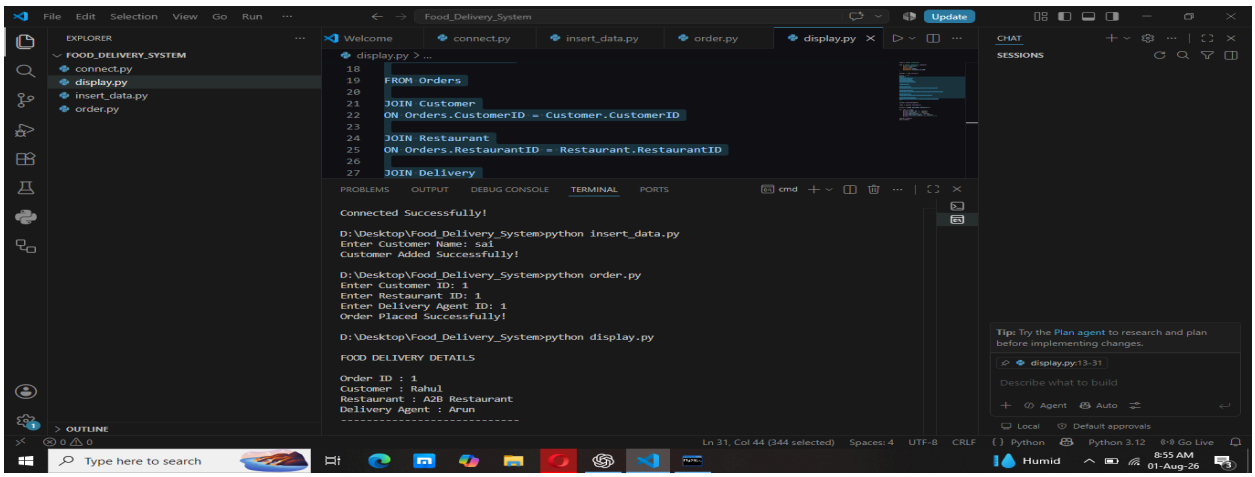
- Create the FoodDeliveryDB database.
- Create Customer, Restaurant, Orders, DeliveryAgent, and Delivery tables.
- Insert sample customer, restaurant, and delivery agent records.
- Connect Python with MySQL.
- Add customer details.
- Place a food order.
- Assign a delivery agent.
- Display food delivery details using SQL JOIN.
- Verify the output.

OUTPUT:




MySQL 9.6 Command Line Client
mysql> SHOW TABLES;
+-----------------------+
| Tables_in_fooddeliverydb |
+-----------------------+
| customer |
| delivery |
| deliveryagent |
| orders |
| restaurant |
+-----------------------+
5 rows in set (0.017 sec)
mysql> INSERT INTO Customer(Name)
-> VALUES
-> ('Rahul'),
-> ('Priya');
Query OK, 2 rows affected (0.136 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO Restaurant(Name)
-> VALUES
-> ('A2B Restaurant'),
-> ('Dominos');
Query OK, 2 rows affected (0.095 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> INSERT INTO DeliveryAgent(Name)
-> VALUES
-> ('Arun'),
-> ('Kumar');
Query OK, 2 rows affected (0.107 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql> select*from Restaurant;
+--------------+---------------+
| RestaurantID | Name |
+--------------+---------------+
| 1 | A2B Restaurant |
| 2 | Dominos |
+--------------+---------------+
2 rows in set (0.018 sec)
mysql> SELECT * FROM Customer;
+------------+-------+
Type here to search
Humid
8:54 AM
01-Aug-26


Select MySQL 9.6 Command Line Client
mysql> SELECT * FROM Customer;
+------------+-------+
| CustomerID | Name |
+------------+-------+
| 1 | Rahul |
| 2 | Priya |
+------------+-------+
2 rows in set (0.005 sec)
mysql> SELECT * FROM DeliveryAgent;
+---------+-------+
| AgentID | Name |
+---------+-------+
| 1 | Arun |
| 2 | Kumar |
+---------+-------+
2 rows in set (0.005 sec)
mysql> SELECT
-> Orders.OrderID,
-> Customer.Name,
-> Restaurant.Name,
-> DeliveryAgent.Name
->
-> FROM Orders
->
-> JOIN Customer
-> ON Orders.CustomerID = Customer.CustomerID
->
-> JOIN Restaurant
-> ON Orders.RestaurantID = Restaurant.RestaurantID
->
-> JOIN Delivery
-> ON Orders.OrderID = Delivery.OrderID
->
-> JOIN DeliveryAgent
-> ON Delivery.AgentID = DeliveryAgent.AgentID;
+---------+-------+---------------+------+
| OrderID | Name | Name | Name |
+---------+-------+---------------+------+
| 1 | Rahul | A2B Restaurant | Arun |
+---------+-------+---------------+------+
1 row in set (0.012 sec)
Type here to search
Humid
8:54 AM
01-Aug-26


File Edit Selection View Go Run
Food_Delivery_System
Update
EXPLORER
FOOD_DELIVERY_SYSTEM
connect.py
display.py
insert_data.py
order.py
Welcome
connect.py
insert_data.py
order.py
display.py
CHAT
SESSIONS
display.py > ...
18 FROM Orders
19
20 JOIN Customer
21 ON Orders.CustomerID = Customer.CustomerID
22
23 JOIN Restaurant
24 ON Orders.RestaurantID = Restaurant.RestaurantID
25
26 JOIN Delivery
27
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cmd
Connected Successfully!
D:\Desktop\Food_Delivery_System>python insert_data.py
Enter Customer Name: sai
Customer Added Successfully!
D:\Desktop\Food_Delivery_System>python order.py
Enter Customer ID: 1
Enter Restaurant ID: 1
Enter Delivery Agent ID: 1
Order Placed Successfully!
D:\Desktop\Food_Delivery_System>python display.py
FOOD DELIVERY DETAILS
Order ID : 1
Customer : Rahul
Restaurant : A2B Restaurant
Delivery Agent : Arun
Tips: Try the Plan agent to research and plan before implementing changes.
display.py:13-31
Describe what to build
Agent Auto
Local Default approvals
OUTLINE
Ln 31, Col 44 (344 selected) Spaces: 4 UTF-8 CRLF Python Python 3.12 Go Live
Type here to search
Humid
8:55 AM
01-Aug-26

Viva Questions and Answers

Q1. Why is Delivery maintained as a separate table?

The Delivery table is kept separate because delivery information is different from order information. It stores the relationship between an order and the delivery agent, reducing redundancy and making it easier to manage delivery assignments.

Q2. How would the schema change if multiple delivery agents could handle a single order?

If multiple delivery agents could handle one order, the Delivery table would allow multiple records for the same OrderID. Instead of making OrderID the primary key, a composite primary key (OrderID, AgentID) or a separate DeliveryID column would be used. This would support a many-to-many relationship between orders and delivery agents.

CONCLUSION: The Food Delivery Application efficiently manages food orders and delivery assignments using a relational database. Python and MySQL provide an effective solution for storing, retrieving, and managing food delivery data.